

Réécriture de code récursif à l'aide de boucles

en langage Caml Light

Présentation de **Lorem Ipsum (50792)**

travail réalisé avec **Dolor sit amet (15980)**

Fonctions récursives et boucles

- ▶ On a souvent besoin d'itérer des instructions sur des données, c'est-à-dire de parcourir une certaine séquence de données en les traitant une par une

Fonctions récursives et boucles

- ▶ On a souvent besoin d'itérer des instructions sur des données, c'est-à-dire de parcourir une certaine séquence de données en les traitant une par une
- ▶ Deux méthodes existent : boucles et fonctions récursives

Boucles

Avantages :

Boucles

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;

Boucles

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;
- ▶ Parfois plus facile à programmer et à lire.

Boucles

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;
- ▶ Parfois plus facile à programmer et à lire.

Exemple :

```
1  let somme_liste (l: int array) =  
2      let resultat = ref 0 in  
3      for i = 0 to Array.length l do  
4          resultat := !resultat + l.(i)  
5      done;  
6      !resultat
```

Fonctions récursives

Avantage : parfois préférable en terme de lisibilité de code.

Fonctions récursives

Avantage : parfois préférable en terme de lisibilité de code.

Problème : peut entraîner des dépassements de pile.

Fonctions récursives

Avantage : parfois préférable en terme de lisibilité de code.

Problème : peut entraîner des dépassements de pile.

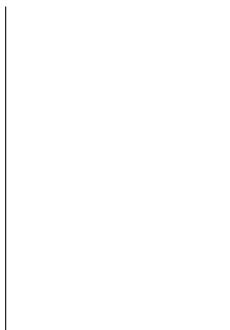
Exemple :

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

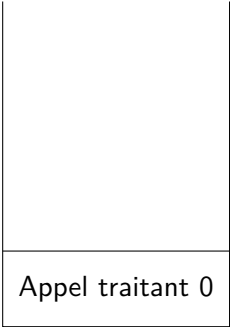
Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



Appel traitant 0

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :

Appel traitant 3
Appel traitant 2
Appel traitant 1
Appel traitant 0

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

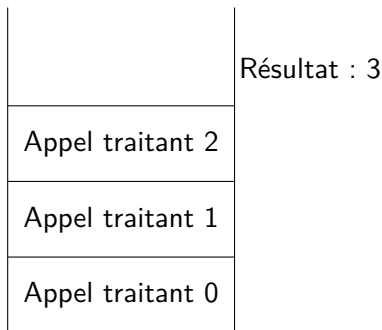
Sur la liste [0; 1; 2; 3] :

	Résultat : 3
Appel traitant 3	
Appel traitant 2	
Appel traitant 1	
Appel traitant 0	

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```

1 | let rec somme_liste (l: int list) =
2 |   match l with
3 |   | [] -> 0
4 |   | x::q -> x + somme_liste q

```

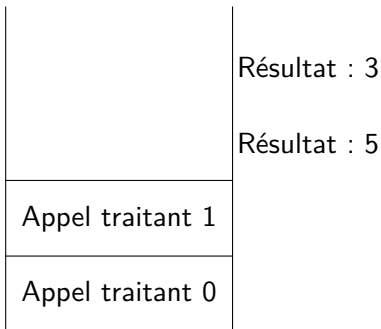
Sur la liste [0; 1; 2; 3] :

	Résultat : 3
Appel traitant 2	Résultat : 5
Appel traitant 1	
Appel traitant 0	

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



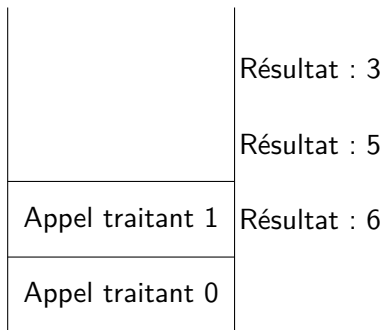
Fonctions récursives

```

1 | let rec somme_liste (l: int list) =
2 |   match l with
3 |   | [] -> 0
4 |   | x::q -> x + somme_liste q

```

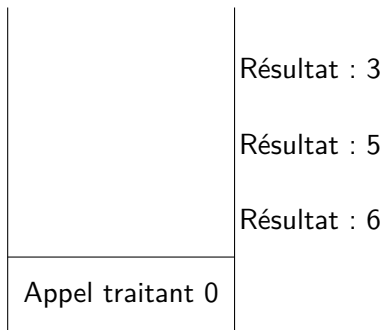
Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :



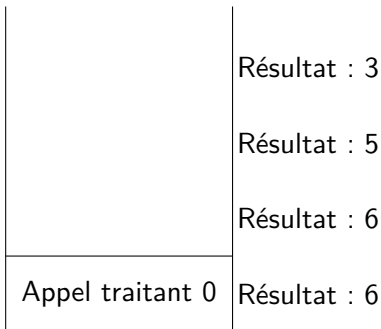
Fonctions récursives

```

1 | let rec somme_liste (l: int list) =
2 |   match l with
3 |   | [] -> 0
4 |   | x::q -> x + somme_liste q

```

Sur la liste [0; 1; 2; 3] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3] :

Résultat : 3

Résultat : 5

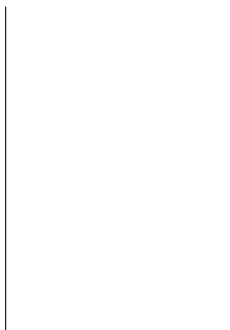
Résultat : 6

Résultat : 6

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :

Appel traitant 0

Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :



Fonctions récursives

```
1 | let rec somme_liste (l: int list) =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Sur la liste [0; 1; 2; 3; 4] :



Passage des fonctions récursives aux boucles

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels
- ▶ Comment ça marche en pratique ?

Passage des fonctions récursives aux boucles

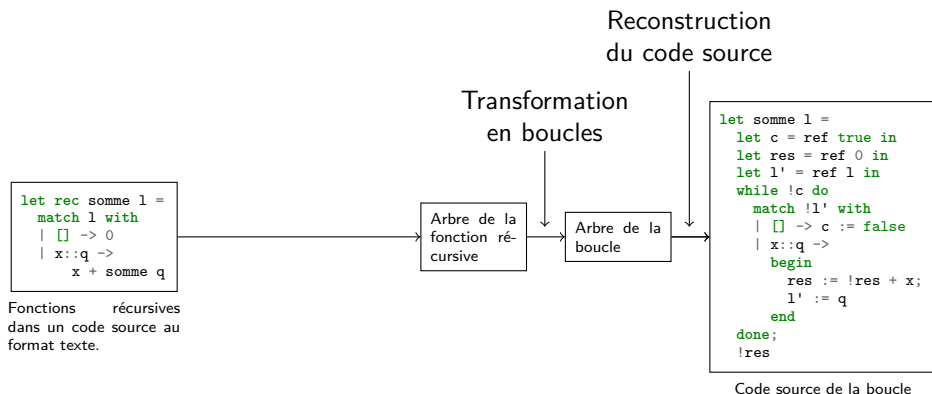
```
let rec somme l =  
  match l with  
  | [] -> 0  
  | x::q ->  
    x + somme q
```

Fonctions récursives
dans un code source au
format texte.

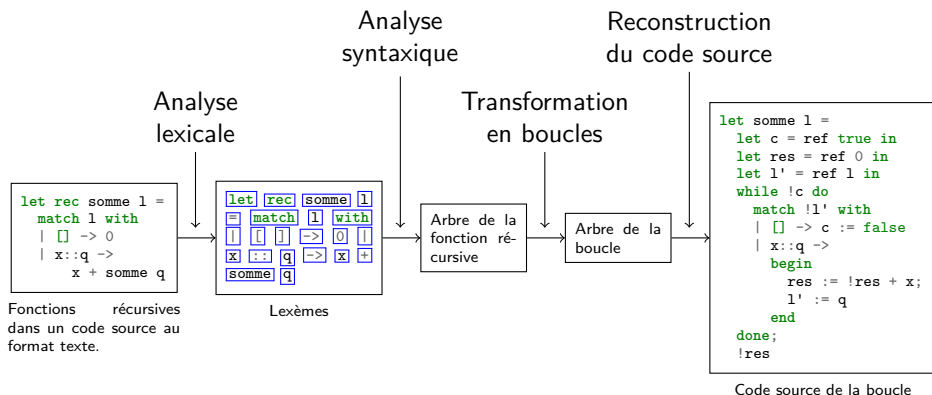
```
let somme l =  
  let c = ref true in  
  let res = ref 0 in  
  let l' = ref l in  
  while !c do  
    match !l' with  
    | [] -> c := false  
    | x::q ->  
      begin  
        res := !res + x;  
        l' := q  
      end  
  done;  
  !res
```

Code source de la boucle

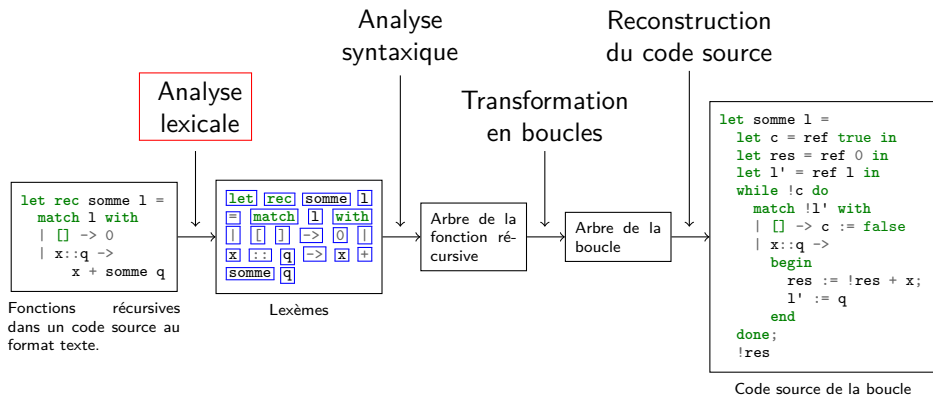
Passage des fonctions récursives aux boucles



Passage des fonctions récursives aux boucles



Plan



Analyse lexicale

But : construire, à partir du code source, une liste de lexèmes plus faciles à traiter que du texte brut.

Analyse lexicale

But : construire, à partir du code source, une liste de lexèmes plus faciles à traiter que du texte brut.

Définition (lexème)

*Un lexème est un couple (**type**, **valeur**). Le **type** doit figurer dans une liste que l'on a prédéfini, et la **valeur** doit correspondre au type tel qu'on l'a prédéfini.*

*Les types de lexèmes sont également appelés **symboles terminaux**.*

Analyse lexicale

But : construire, à partir du code source, une liste de lexèmes plus faciles à traiter que du texte brut.

Définition (lexème)

*Un lexème est un couple (**type**, **valeur**). Le **type** doit figurer dans une liste que l'on a prédéfini, et la **valeur** doit correspondre au type tel qu'on l'a prédéfini.*

*Les types de lexèmes sont également appelés **symboles terminaux**.*

Exemples : (int, 12), (string, "Hello, world"), (equals, =).

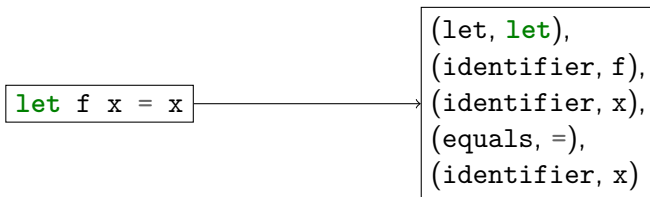
Analyse lexicale

Exemple :

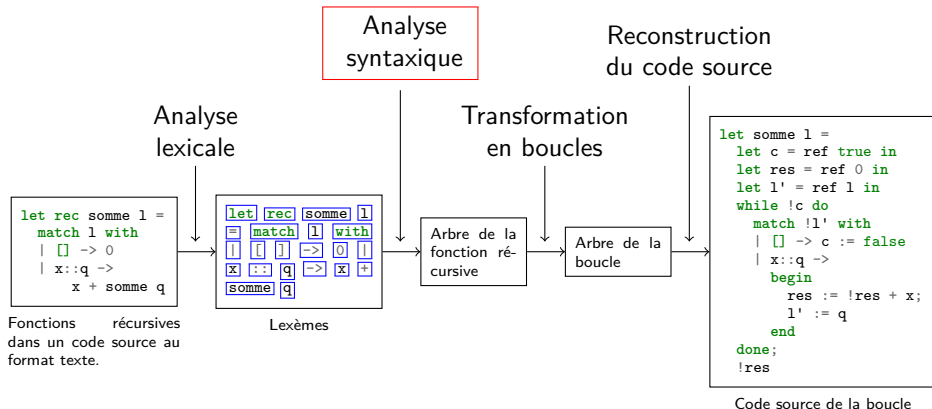
```
let f x = x
```


Analyse lexicale

Exemple :



Plan



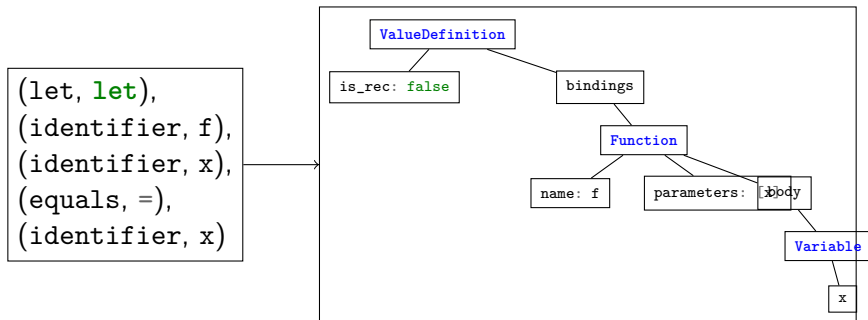
Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Exemple :



Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Grammaires

Pour décrire les « règles de branchement » dans nos arbres, on introduit la notion de grammaire.

Définition (grammaire)

*Une grammaire est un ensemble de règles (dites « règles de dérivation ») de la forme $\boxed{\text{Symbole} \rightarrow \dots}$. Les symboles pouvant être mis à gauche des flèches sont appelés **symboles non terminaux**. À droite des flèches apparaissent une liste de **symboles terminaux et non terminaux** (les symboles terminaux étant des types de lexèmes).*

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple de texte :

$(3 + (1 + 2)) = (3 \times 2)$

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple de texte :

$$(3 + \underbrace{(1 + 2)}_{\text{EXPR}}) = (3 \times 2)$$

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple de texte :

$$\overbrace{(3 + \underbrace{(1 + 2)}_{\text{EXPR}})}^{\text{EXPR}} = (3 \times 2)$$

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple de texte :

$$\overbrace{(3 + \underbrace{(1 + 2)}_{\text{EXPR}})}^{\text{EXPR}} = \overbrace{(3 \times 2)}^{\text{EXPR}}$$

Exemple

PHRASE $\rightarrow$ $\text{EXPR} = \text{EXPR}$

$\text{EXPR} \rightarrow \text{integer_literal}$

$\text{EXPR} \rightarrow (\text{EXPR plus EXPR})$

$\text{EXPR} \rightarrow (\text{EXPR times EXPR})$

Exemple de texte :

$$\underbrace{\overbrace{(3 + \underbrace{(1 + 2)}_{\text{EXPR}})}^{\text{EXPR}} = \overbrace{(3 \times 2)}^{\text{EXPR}}}_{\text{PHRASE}}$$

